# The pxTAB package

Predefined table styles and list-driven tables for LaTeX2e

Scvria
P3CTeX / UOC PEC Repository

2026/03/02 – Version 0.1

**Abstract**

pxTAB provides a small LaTeX2e API to build tables from comma-separated rows or list-style data without writing raw tabular environments. It offers a handful of predefined visual styles (plain, header, striped, boxed, grid) and a set of composed header styles (header-boxed, header-grid, header-striped), together with key–value options for consistent layout control.

## Contents

**1 Loading the package** **2**

**2 Configuration (keys)** **2**

**3 Styles by example** **6**
- 3.1 Plain . . . 6
- 3.2 Header . . . 7
- 3.3 Striped . . . 7
- 3.4 Boxed . . . 8
- 3.5 Grid . . . 8
- 3.6 Composed header styles . . . 8
- 3.7 Paragraph cells . . . 10
- 3.8 Centered and floating tables . . . 11

**4 Command reference** **12**
- 4.1 \pxTABsetup . . . 12
- 4.2 \pxTable . . . 13
- 4.3 \pxTableFromList . . . 13
- 4.4 \pxTableRow . . . 13
- 4.5 \pxTableHeadBody . . . 13
- 4.6 \pxTableHeadBodyRows . . . 14
- 4.7 \pxTableMatrix . . . 14
- 4.8 Quick reference . . . 14

**5 Advanced usage** **15**
5.1 Best practices . . . . . 15
5.2 When to use paragraph mode . . . . . 16
5.3 Explicit column specifications . . . . . 16
5.4 Width control . . . . . 17
5.5 Font size and row spacing . . . . . 17
5.6 Troubleshooting . . . . . 17

**6 Implementation notes** **18**

**7 Integration with P3CTeX** **19**

# 1 Loading the package

```
\usepackage{pxTAB}
```

Package options are forwarded to the internal key family `pxTAB` and are equivalent to calling `\pxTABsetup` in the preamble:

```
\usepackage[style=header,headerbgcolor=blue!15]{pxTAB}
```

# 2 Configuration (keys)

The `pxTAB` key family controls both visual style and parsing behaviour. Keys can be set at load time or via:

```
\pxTABsetup{<key>=<value>,...}
```

Defaults are chosen to be reasonable for course work and reports. All keys affect subsequent tables until changed again (usual LaTeX scoping rules apply).

## Style and colours

- `style` (choice, default `plain`)
  Selects the high-level style: `plain`, `header`, `striped`, `boxed`, `grid`, or one of the composed header styles `header-boxed`, `header-grid`, `header-striped`. The composed styles behave like the corresponding base style (boxed, grid, striped) with an additional coloured header row.

- `headerbgcolor` (token list, default `gray!25`)
  Background colour of the header row in `header` style.

- `headertextcolor` (token list, default `black`)
  Text colour used in the header row.

- `altrowbgcolor` (token list, default `gray!10`)
  Background colour for alternating rows in `striped` style.
- `altrowcolor` (meta)
  Alias for `altrowbgcolor`.
- `altrowtextcolor` (token list, default empty)
  Text colour used on alternating rows in `striped` style.
- `headerfont` (token list, default `\bfseries`)
  Font-switch applied to the header row (e.g. `\sffamily\bfseries`).
- `bodyfont` (token list, default empty)
  Font-switch applied to body rows.
- `rulecolor` (token list, default `black!55`)
  Colour used for horizontal and vertical rules (applied via `\arrayrulecolor`).
- `rulewidth` (dimension, default `0.4pt`)
  Thickness of rules (applied via `\arrayrulewidth`).

### Geometry and layout

- `tabcolsep` (dimension, default `6pt`)
  Column separation (mirrors the standard `\tabcolsep` parameter).
- `arraystretch` (token list, default `1.2`)
  Row stretch applied via `\arraystretch`.
- `rowstretch` (meta)
  Alias for `arraystretch`.
- `width` (code, default `natural`)
  Width of the table. The default `natural` keeps the table at its natural (fit-content) width; tables are *not* stretched to page width unless you explicitly set `width` or `tabwidth`. The special values `linewidth` and `line-width` are treated as `\linewidth`; any other value is used as-is (e.g. `.8\linewidth` or `120pt`).
- `tabwidth` (meta)
  Alias for `width`.
- `centering` (boolean, default `false`)
  When `true`, inserts a `\centering` before the internal `tabular` or `tabular*`. This only changes the horizontal alignment of the table block (in a paragraph or inside a `table` float); it does not modify the computed width. In particular, `width=\linewidth` still produces a full-width table, but the block is explicitly centred. The key can be set

globally with `\pxTABsetup` or per table in the optional `[<options>]` argument of the commands below.

- `fontsize` (token list, default empty)
  Optional font-size command (such as `\small` or `\footnotesize`) applied to the table.

- `size` (meta)
  Alias for `fontsize`.

- `align` (choice, default `c`)
  When `colspec` is not provided, controls automatic column alignment: `l`, `c`, or `r`.

- `alignment` (meta)
  Alias for `align`.

- `colspec` (token list, default empty)
  Explicit `tabular` column specification (e.g. `ll`, `|c|c|`). When omitted, `pxTAB` builds one automatically from `align`. Simple `l`/`c`/`r` specifications are checked against the inferred column count; on mismatch a warning is issued and an automatic specification is used.

### Floats and captions

The following keys enable a lightweight float wrapper around any `pxTAB` table command. They can be set at package level or passed in the per-table options:

- `caption` (token list, default empty)
  When non-empty, the table body is wrapped in a `table` environment and `\caption{<caption>}` is emitted above the tabular.

- `label` (token list, default empty)
  Optional label placed immediately after the caption (or, if there is no caption, at the top of the float body). A recommended convention is to use prefixes such as `tab:grades` so that `\ref{tab:grades}` yields the table number in the usual way.

- `float` (token list, default empty)
  Placement specifier forwarded to the `table` environment (e.g. `htbp`). When empty, LaTeX uses its default float placement rules.

Any non-empty combination of `caption`, `label`, or `float` activates the float wrapper; otherwise tables are generated in-place without a surrounding `table` environment. When both `centering=true` and float keys are used, the `\centering` is inserted inside the `table` environment, before the caption and the tabular body.

### Parsing

- `rowsep` (token list, default `;`)
  Separator between rows when using list-style data.

- `cellsep` (token list, default `,`)
  Separator between cells inside each row.

- `probe` (boolean, default `false`)
  When `true`, writes diagnostic information to the LaTeX log file during table rendering (style name, colspec, visual settings, etc.). Intended for debugging and development; no effect on output. QA harnesses may override the internal probe hook for automated assertions.

- `unknown` (code)
  Unknown keys are currently ignored silently (reserved for future extension).

### Paragraph cells

By default `pxTAB` produces *inline* columns (`l`/`c`/`r`), suitable for short labels and numbers. Paragraph mode enables fixed-width columns that wrap multi-line content, mirroring the equal-width paragraph layout previously available through `pxTBL`.

- `cellmode` (choice, default `inline`)
  Controls the column type used by the automatic colspec builder:

  - `inline` — standard behaviour; columns are simple alignment columns derived from `align` (`l`/`c`/`r`).
  - `paragraph` — each column becomes a paragraph column (`p{...}` or `m{...}`) with equal width, so cell content wraps across multiple lines.

  Paragraph mode is *opt-in*; existing documents that do not set this key are unaffected.

- `paragraphcells` (boolean, default `true` when used)
  Convenience alias: `paragraphcells=true` is equivalent to `cellmode=paragraph`, and `paragraphcells=false` to `cellmode=inline`. When used without an explicit value (bare `\pxTABsetup{paragraphcells}`) it defaults to `true`.

- `paragraphcoltype` (choice, default `p`)
  Selects the underlying LaTeX column type used in paragraph mode:

  - `p` — `p{<width>}` columns; cell content is top-aligned within the cell box.

– `m` — `m{<width>}` columns (requires the `array` package, loaded automatically by `pxTAB`); cell content is vertically centred within the cell box.

This key is ignored when `cellmode=inline`.

**Effective width.** When paragraph mode is active the per-column width is computed as

$$w_{\text{col}} = \frac{W_{\text{eff}}}{n_{\text{cols}}}$$

where $W_{\text{eff}}$ is the *effective table width*:

- If `width` (or `tabwidth`) is set to an explicit dimension (e.g. `0.8\linewidth`, `120pt`), that value is used directly as $W_{\text{eff}}$.
- If `width=natural` (the default), paragraph mode uses `\linewidth` as $W_{\text{eff}}$, since paragraph columns require an explicit dimension. The resulting table occupies the full text width by default.

Horizontal alignment inside each paragraph cell is governed by the `align` key: `l` maps to `\raggedright`, `c` to `\centering`, and `r` to `\raggedleft`.

**Interaction with `colspec`.** When the user provides an explicit `colspec`, it is used as-is regardless of `cellmode`. In other words, `cellmode=paragraph` only affects the *automatic* colspec builder; advanced users who need heterogeneous column widths or mixed column types should set `colspec` directly.

# 3 Styles by example

This section illustrates the built-in styles using a small example dataset. In all cases we use:

```
\pxTABsetup{tabwidth=.8\linewidth}
```

## 3.1 Plain

```
\pxTABsetup{style=plain,tabwidth=.8\linewidth}
\pxTable{Name, Age, City}
        {Alice, 20, Barcelona}
        {Bob, 22, Madrid}
```

| Name | Age | City |
|---|---|---|
| Alice | 20 | Barcelona |
| Bob | 22 | Madrid |

### 3.2 Header

```
\pxTABsetup
  {
    style=header,
    headerbgcolor=blue!15,
    headertextcolor=blue!70!black,
    headerfont=\sffamily\bfseries
  }
\pxTable{Name, Age, City}
        {Alice, 20, Barcelona}
        {Bob, 22, Madrid}
```

| Name | Age | City |
|---|---|---|
| Alice | 20 | Barcelona |
| Bob | 22 | Madrid |

### 3.3 Striped

```
\pxTABsetup
  {
    style=striped,
    altrowbgcolor=gray!10,
    altrowtextcolor=black
  }
\pxTable{Item, Quantity}
        {Apples, 3}
        {Oranges, 5}
        {Bananas, 2}
```

| Item | Quantity |
|---|---|
| Apples | 3 |
| Oranges | 5 |
| Bananas | 2 |

In the `striped`, `header-striped`, and `header-grid` styles, row backgrounds are applied with `\rowcolor` so that the colour fills the *entire cell area* across all columns—including any wrapped lines in paragraph mode. This holds for both natural-width and explicit-width tables.

### 3.4 Boxed

```
\pxTABsetup{style=boxed}
\pxTableFromList
  {Code, Description;
   PX1, First code;
   PX2, Second code}
```

| Code | Description |
|---|---|
| PX1 | First code |
| PX2 | Second code |

### 3.5 Grid

```
\pxTABsetup{style=grid}
\pxTableMatrix{2}{3}
  {m11,m12,m13,
   m21,m22,m23}
```

| | | |
|---|---|---|
| m11 | m12 | m13 |
| m21 | m22 | m23 |

### 3.6 Composed header styles

The composed styles combine the header row styling with the boxed, grid, or striped variants so that common table layouts can be selected with a single option. The following examples use the same small dataset.

**`style=header-boxed`**

```
\pxTABsetup
  {
    style=header-boxed,
    headerbgcolor=blue!15,
    headertextcolor=blue!70!black,
    headerfont=\sffamily\bfseries,
    tabwidth=.8\linewidth
  }
\pxTable{Name, Age, City}
        {Alice, 20, Barcelona}
        {Bob, 22, Madrid}
```

| Name | Age | City |
|---|---|---|
| Alice | 20 | Barcelona |
| Bob | 22 | Madrid |

**style=header-grid**

```
\pxTABsetup
  {
    style=header-grid,
    headerbgcolor=blue!15,
    headertextcolor=blue!70!black,
    headerfont=\sffamily\bfseries,
    tabwidth=.8\linewidth
  }
\pxTable{Name, Age, City}
        {Alice, 20, Barcelona}
        {Bob, 22, Madrid}
```

| Name | Age | City |
|---|---|---|
| Alice | 20 | Barcelona |
| Bob | 22 | Madrid |

**style=header-striped**

```
\pxTABsetup
  {
    style=header-striped,
    headerbgcolor=blue!15,
    headertextcolor=blue!70!black,
    headerfont=\sffamily\bfseries,
    altrowbgcolor=gray!10,
    altrowtextcolor=black,
    tabwidth=.8\linewidth
  }
\pxTable{Name, Age, City}
        {Alice, 20, Barcelona}
        {Bob, 22, Madrid}
        {Clara, 23, Girona}
```

| Name | Age | City |
|---|---|---|
| Alice | 20 | Barcelona |
| Bob | 22 | Madrid |
| Clara | 23 | Girona |

### 3.7 Paragraph cells

Paragraph mode produces equal-width columns that wrap long text, suitable for rubrics, notes, and narrative content. The following example uses three columns with `cellmode=paragraph` and a fixed table width:

```
\pxTABsetup
  {
    style=plain,
    cellmode=paragraph,
    paragraphcoltype=p,
    width=.8\linewidth
  }
\pxTable
  {Topic, Description, Status}
  {Design, The API was specified using key-value options
   to keep the interface consistent with existing pxTAB
   commands., Complete}
  {Tests, Paragraph regression tests exercise multi-line
   wrapping and interaction with all styles., Passed}
```

| Topic | Description | Status |
|---|---|---|
| Design | The API was specified using key-value options to keep the interface consistent with existing pxTAB commands. | Complete |
| Tests | Paragraph regression tests exercise multi-line wrapping and interaction with all styles. | Passed |

Paragraph mode works with all styles. The next example combines `header-striped` with vertically-centred `m` columns:

```
\pxTABsetup
  {
    style=header-striped,
    paragraphcells=true,
    paragraphcoltype=m,
    width=.85\linewidth,
    headerbgcolor=blue!12,
    headertextcolor=blue!70!black,
    altrowbgcolor=gray!10,
    altrowtextcolor=black
  }
\pxTable{Criterion, Comment}
  {Readability, This rubric comment wraps inside
   a paragraph cell while striped backgrounds fill
   the entire cell area across all wrapped lines.}
  {Correctness, Another multi-line comment to show
   that composed styles and paragraph columns work
   together seamlessly.}
```

| Criterion | Comment |
|---|---|
| Readability | This rubric comment wraps inside a paragraph cell while striped backgrounds fill the entire cell area across all wrapped lines. |
| Correctness | Another multi-line comment to show that composed styles and paragraph columns work together seamlessly. |

### 3.8 Centered and floating tables

The new `centering` and float-related keys can be combined with any of the predefined styles. The following example uses `style=header` with global centering:

```
\pxTABsetup
  {
    style=header,
    headerbgcolor=blue!15,
    headertextcolor=blue!70!black,
    headerfont=\sffamily\bfseries,
    centering=true
```

```
  }
\pxTable{Name, Age, City}
        {Alice, 20, Barcelona}
        {Bob, 22, Madrid}
```

| Name | Age | City |
|---|---|---|
| Alice | 20 | Barcelona |
| Bob | 22 | Madrid |

A float table with caption and label can be created by passing the new keys in the options argument:

```
\pxTable[
  style=header,
  caption={Sample grades},
  label=tab:grades,
  float=htbp
]{Name, Grade}
 {Alice, 9}
 {Bob, 7}
```

Table 1: Sample grades

| Name | Grade |
|---|---|
| Alice | 9 |
| Bob | 7 |

As usual, the table can later be referenced with `Table~\ref{tab:grades}` in the surrounding text.

# 4 Command reference

This section summarises the user-facing commands provided by `pxTAB`. All commands respect the key settings described above.

## 4.1 `\pxTABsetup`

```
\pxTABsetup{<key>=<value>,...}
```

Sets package-wide defaults in the `pxTAB` key family. Usual LaTeX scoping rules apply, so placing a setup call inside a group affects only the tables in that group.

### 4.2 \pxTable

```
\pxTable[<options>]{<row1>}{<row2>}...{<rowN>}
```

Builds a table from braced row arguments. Each `<row>` is a comma-separated list of cell contents. A typical example is:

```
\pxTABsetup{style=header}
\pxTable{Name, Age, City}
        {Alice, 20, Barcelona}
        {Bob, 22, Madrid}
```

### 4.3 \pxTableFromList

```
\pxTableFromList[<options>]{<data>}
```

Builds a table from a single list-style string. By default rows are separated by semicolons (`;`) and cells by commas (`,`):

```
\pxTableFromList
  {Name, Age, City;
   Alice, 20, Barcelona;
   Bob, 22, Madrid}
```

The separators can be customised with the `rowsep` and `cellsep` keys.

### 4.4 \pxTableRow

```
\pxTableRow[<options>]{<row>}
```

Convenience wrapper for single-row tables (headers or small data rows). It behaves as:

```
\pxTableRow[<opts>]{<row>} == \pxTable[<opts>]{<row>}
```

### 4.5 \pxTableHeadBody

```
\pxTableHeadBody[<options>]{<header>}{<body-list>}
```

The header row is given explicitly as a comma-separated list, while the body is provided as list-style data using the current separators:

```
\pxTableHeadBody
  {Name, Role}
  {Alice, Dev; Bob, QA; Carol, PO}
```

### 4.6 \pxTableHeadBodyRows

```
\pxTableHeadBodyRows[<options>]
  {<header>}{<row1>}{<row2>}...
```

Variant of the previous command where both header and body rows are given as separate braced arguments:

```
\pxTableHeadBodyRows
  {Name, Score}
  {Anna, 8}
  {Marc, 9}
  {Joan, 10}
```

### 4.7 \pxTableMatrix

```
\pxTableMatrix[<options>]{<rows>}{<cols>}{<flat-cells>}
```

Builds a table from a flat list of cells, filled row by row. Missing cells are left blank:

```
\pxTableMatrix{2}{3}
  {m11,m12,m13,
   m21,m22,m23}
```

### 4.8 Quick reference

The following table summarises the most common commands and their arguments; all of them honour the global and per-table keys described in the configuration section.

| Command | Arguments | Typical use |
|---|---|---|
| \pxTABsetup | {<key>=<value>,...} | Set global defaults ( centering, colours, fl |
| \pxTable | [<options>]{<row1>}{...}{<rowN>} | Small to medium tered as explicit row centering, capti float. |
| \pxTableFromList | [<options>]{<data>} | Tables given as a string separated by cellsep. |
| \pxTableRow | [<options>]{<row>} | Single-row tables quick inline tables). |
| \pxTableHeadBody | [<options>]{<header>}{<body-list>} | Head/body split wi given as list-style dat |
| \pxTableHeadBodyRows | [<options>]{<header>}{<row1>}{...} | Head/body split header and body ro explicitly. |
| \pxTableMatrix | [<options>]{<rows>}{<cols>}{<flat-cells>} | Matrix-style data wh list is reshaped into a |

The float-related keys `caption`, `label`, and `float`, together with `centering`, can be provided either globally via `\pxTABsetup` or per table in the `[<options>]` argument.

# 5 Advanced usage

## 5.1 Best practices

- Use `style=plain` for compact, in-text tables where visual emphasis is not required; `style=header` works well for small datasets with a clear header row.
- For longer datasets, `style=striped` improves readability, while `boxed` and `grid` are suitable when a strong tabular structure is desired (summaries, rubrics, code lists).
- Prefer the natural-width default together with `centering=true` when the table is narrower than the text block; this keeps columns tight while visually centring the table.
- Use `width=\linewidth` (or a fraction thereof) when the table should align with surrounding paragraphs; in this case `centering` is optional but harmless.

- When using floats, set both `caption` and `label` so that tables integrate with the document's list of tables and can be referenced with `\ref`.

## 5.2 When to use paragraph mode

Paragraph mode (`cellmode=paragraph`) is designed for tables whose cells contain narrative text, rubric comments, or any content that benefits from line wrapping:

- **Prefer paragraph mode** when one or more columns hold multi-line descriptions, notes, or explanatory text. Typical examples include rubric tables, requirement matrices, and coursework summary tables.
- **Prefer inline mode** (the default) when all columns contain short labels, numbers, or single-line entries. Inline columns produce tighter, more compact tables.
- **Width interaction.** Paragraph mode always requires an effective width to compute per-column dimensions. When `width=natural` (the default), `\linewidth` is used automatically; for narrower tables set an explicit width such as `width=0.8\linewidth`. The `centering` key works identically in paragraph and inline mode—it controls horizontal placement of the table block, not the column widths.
- **Vertical alignment.** Use `paragraphcoltype=p` (default) for top-aligned cells or `paragraphcoltype=m` when rows of unequal height should be vertically centred.

**Known limitations.**

- Paragraph mode produces *equal-width* columns only; heterogeneous widths require a manual `colspec`.
- An explicit `colspec` overrides the automatic paragraph layout entirely—users who set `colspec` are responsible for choosing appropriate `p{...}` or `m{...}` column types.
- There is no `longtable` integration; page-breaking paragraph tables remain the domain of `pxTBL`'s `\pxTBLlong` command or manual `longtable` usage.

## 5.3 Explicit column specifications

When `colspec` is set, its value is used directly as the `tabular` column specification:

```
\pxTABsetup{style=plain,colspec=ll}
\pxTable{Col 1, Col 2}
        {Left, also left}
```

Simple `l`/`c`/`r` specifications are checked against the inferred column count; on mismatch a warning is issued and an automatic specification is used.

### 5.4 Width control

The `width`/`tabwidth` keys allow fixed-width tables:

```
\pxTABsetup{style=plain,width=\linewidth,align=l}
\pxTable{A, B, C}{1, 2, 3}{4, 5, 6}
```

Non-`natural` widths use a `tabular*` layout with automatic extra column separation so that the table fits the requested width.

### 5.5 Font size and row spacing

```
\pxTable[fontsize=\small,rowstretch=1.05,style=header]
        {Name, Notes}
        {Alice, Compact rows with smaller text}
        {Bob, This checks per-table sizing controls}
```

The `fontsize`/`size` and `arraystretch`/`rowstretch` keys apply only to the current table.

### 5.6 Troubleshooting

- *Empty or missing tables.* If the first row has no cells, `pxTAB` skips the table and issues a warning. Check that your header row is non-empty and that the separators (`rowsep`, `cellsep`) are correct.
- *Column specification mismatch.* When `colspec` is given but its length does not match the inferred column count, an automatic specification is used instead and a warning is written to the log. Adjust `colspec` or let `pxTAB` derive it from `align`.
- *Float placement warnings.* The `float` key is passed directly to `\begin{table}[...]`. If you use an unsupported placement specifier, LaTeX may report an "Unknown float option" warning; choose a standard value such as `h`, `t`, or `htbp`.
- *Missing caption or label.* A float without `label` still typesets correctly but cannot be referenced with `\ref`. Conversely, a `label` without `caption` is allowed; the table is still wrapped in a `table` environment so that the label has a proper float context.

# 6 Implementation notes

This subsection describes, at a high level, how `pxTAB` processes table data and renders it. It is intended for maintainers and advanced users who need to understand the internal pipeline.

### Row parsing and trailing empty trimming

Input data (from `\pxTable`, `\pxTableFromList`, or the head/body helpers) is stored as a sequence of rows in `\l__pxtab_rows_seq`. For list-style input, rows are split using the `rowsep` separator (default `;`), and cells within each row using `cellsep` (default `,`).

Before rendering, `\__pxtab_trim_trailing_empty_rows:` recursively removes trailing rows that have no non-blank cells. A row is considered empty if every cell, after splitting by `cellsep`, is blank. This ensures that trailing semicolons (e.g. `A,B;C,D;`) do not produce extra blank rows in the output.

### Column count and colspec derivation

The column count is taken from the *first* row only: it is the number of cells after splitting by `cellsep`. If the first row is empty, a warning is emitted and the table is skipped.

The column specification (`colspec`) is determined as follows:

- If the user provided `colspec` and it is a simple `l/c/r` string whose length matches the inferred column count, it is used as-is.
- If the provided `colspec` length does not match, a `colspec-mismatch` warning is issued and an automatic colspec is built from the `align` key.
- If no `colspec` was given, one is built automatically by repeating the `align` character (default `c`) for each column.

For `boxed` and `grid` styles, vertical rules are added around the colspec (e.g. `|c|c|c|`).

### Style renderers and shared helpers

Each style has a dedicated renderer (`\__pxtab_render_table_plain:n`, `header:n`, `striped:n`, `boxed:n`, `grid:n`). All renderers share:

- `\__pxtab_append_row_to_body:nnnn` — appends a row of cells to the body token list, with optional text colour and font.

- `\__pxtab_append_row_styled_to_body:nnnnn` — same, plus optional `\rowcolor` for background.
- `\__pxtab_emit_tabular_with_body:N` — applies visual setup (`tabcolsep`, `arraystretch`, `rulecolor`, etc.) and emits the `tabular` or `tabular*` with the accumulated body.

Style-specific behaviour:

- `plain`: all rows use body font; no row colours or rules.
- `header`: first row gets `headerbgcolor`, `headertextcolor`, `headerfont`; body rows use body font.
- `striped`: odd rows plain; even rows get `altrowbgcolor` and `altrowtextcolor`. Backgrounds are applied via `\rowcolor` so they fill the entire cell area (including wrapped lines in paragraph mode), not just the text extent.
- `boxed`: colspec wrapped with `|`; `\hline` before first row and after last row.
- `grid`: colspec wrapped with `|`; `\hline` before and after every row.
- `header-boxed`: header-style first row (colours and font) with boxed frame rules as in `boxed`.
- `header-grid`: header-style first row together with full grid rules as in `grid`. The header row background uses `\rowcolor`, filling the entire cell area.
- `header-striped`: header-style first row; subsequent body rows alternate plain and striped colours like `striped`, with the striping parity starting at the first body row. As with `striped`, row backgrounds fill the full cell width across all columns and wrapped lines.

# 7 Integration with P3CTeX

When using the `P3CTeX` document class and its core package `pxCORE`, the `pxTAB` package is loaded as part of the default configuration so that table commands are available without extra setup. The example document `P3CTeX-example.tex` in the repository includes several small `pxTAB` tables alongside the other P3CTeX modules.